**CS319: Scientific Computing**

# Multidimensional Arrays; Introduction to Classes

Dr Niall Madden

Week 7: 25 + 27 February, 2026

Slides and examples: https://www.niallmadden.ie/2526-CS319

# 0. Notices: grades and git

1. Sorry - still have not graded Lab 2. Soon! **Done!!!**
2. Will also grade the Class Test by next week.
3. Grades for Lab 4: will be confirmed once demo'ed in a lab. **You need to attend a lab to get a non-zero grade**.

. . . . . .

Want to quickly download all the C++ examples?

- Install `git`: see `https://git-scm.com/install/`. Note: if using Linux or Mac, it may already be installed, and if not use `apt` or `brew`, respectively.
- Navigate to a folder where you would like the code to be kept.
- From a terminal, try `git clone https://github.com/niallmadden/2526-CS319.git`
- After that, from the same folder, try `git pull` to get updates and fixes.

# 0. Schedule for remainder of semester

**Schedule for the remainder of the semester**

- Lab 4 due tomorrow. OK?
- Lab 5: Starts Week 7; progress to be uploaded by end Week 7.
- Lab 6: Starts Week 8; code and report uploaded by Wednesday Week 9.
- Projects begin Week 8. Topic selected by Wed of Week 9
- Lab 7: Starts Week 9; progress to be uploaded by end Week 9.
- Project proposals due Wed Week 10
- Lab 8: Starts Week 10; code and report uploaded by end Week 11.
- Project presentations in Week 12: Wed 4-5 and Thursday 9-10.
- Project report and code due Friday 10th April.

# 0. Outline


1. Two-dimensional arrays
   - Recall: 1D
   - 2D arrays
   - 2D DMA
   - Deallocation
2. Quadrature in 2D
   - Trapezium Rule in 2D
3. Preview of Labs 5 and 6
4. Encapsulation
   - Terminology
   - `public` V `private`
5. `class`
   - Example – `MyStack` (V1.0)
   - Syntax: `class`
6. Constructors
   - `MyStack` (Version 2)
   - Destructors
7. `MyStack` (Final version)
   - New class definition
   - Constructors
   - The destructor


Slides and examples:
www.niallmadden.ie/2526-CS319

# 1. Two-dimensional arrays

Recall: 1D

So far in CS319, we have worked with **one-dimensional** arrays. For example, if we wanted to store a set of **five** `int`egers, we could declare an array:

```
int v[5];
```

We could then access the five elements:
`v[0]`, `v[1]`, `v[2]`, `v[3]`, and `v[4]`.

This is a **one-dimensional array**: the array has a single index. It is similar to the idea of a **vector** in Mathematics.

However, often we will have table/rectangles of data, in a way that is similar to a **matrix**.

In C++, a two-dimensional $M \times N$ array of (say) `double`s can be declared as:

```
double A[M][N];
```

A[i,j] ≠ A[i][j]

Then its members are

$$\begin{pmatrix} \text{A[0][0]} & \text{A[0][1]} & \text{A[0][2]} & \cdots & \text{A[0][N-1]} \\ \text{A[1][0]} & \text{A[1][1]} & \text{A[1][2]} & \cdots & \text{A[1][N-1]} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ \text{A[M-1][0]} & \text{A[M-1][1]} & \text{A[M-1][2]} & \cdots & \text{A[M-1][N-1]} \end{pmatrix}$$

For example, `double A[3][4];`
gives a 2D array

$$\begin{pmatrix} \text{A[0][0]} & \text{A[0][1]} & \text{A[0][2]} & \text{A[0][3]} \\ \text{A[1][0]} & \text{A[1][1]} & \text{A[1][2]} & \text{A[1][3]} \\ \text{A[2][0]} & \text{A[2][1]} & \text{A[2][2]} & \text{A[2][3]} \end{pmatrix}$$

Dynamic Memory Allocation in 2D is a little complicated, because a 2D array is actually just an "array of arrays".

DMA

This is because, we declare, for example, `double A[3][4];` what really happens is:

- `A` is assigned the base address of three **pointers**: `A[0]`, `A[1]`, `A[2]`.
- Each of those is a base address for a (1D) array of 4 doubles.

This approach has advantages: because of it the language can support arrays in as many dimensions as one would like.

But it makes DMA more complicated.

To use dynamic memory allocation to reserve memory for a two-dimensional $M \times N$ matrix of doubles (for example):

- declare a "pointer to pointer to double"
- use `new` to assign memory for $M$ pointers;
- for each of those, assign memory for $N$ doubles.

Code:

```
double **A;
A = new double* [M];
for (int i=0; i<M; i++)
        A[i] = new double [N];
```

Reserve memory for each row

If we dynamically allocate memory for a 2D array, we need to de-allocate it too, using the `delete` operator (See Week 6).

If the array `A` as been allocated as on the previous slide, it is de-allocated as:

```
for (int i=0; i<M; i++)
      delete[] A[i];
delete[] A;
```

# 2. Quadrature in 2D

For the last time (in lectures) we'll look at **numerical integration**, this time of two dimensional functions.

That is, our goal is to estimate

$$\int_{a_1}^{b_1} \int_{a_2}^{b_2} f(x_1, x_2) dx_2 dx_1.$$

When we implement an algorithm for this, we will set

- `x1` and `x2` to be vectors of (one-dimensional) quadrature of $N + 1$ points.
- `y` to be a **two-dimensional** array of $(N + 1)^2$ quadrature values. That is, we will set
  `y[i][j] = f(x1[i], x2[j]);`

## Derivation

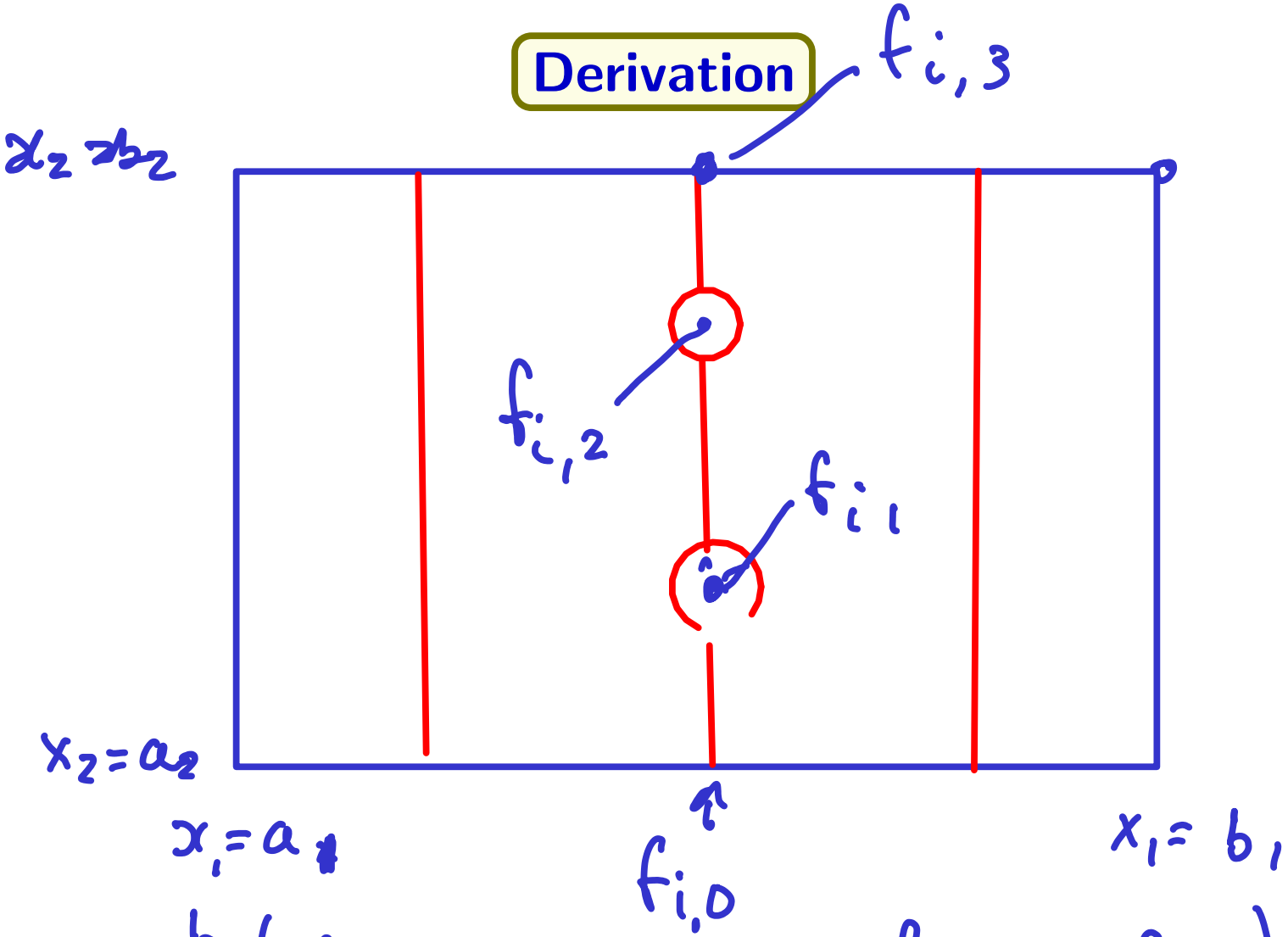
Since $\int_{a_1}^{b_1}\left(\int_{a_2}^{b_2} f(x_1,x_2)\,dx_2\right)dx_1$ can be evaluated in 2 steps

- first integrate with respect to $x_2$
- then integrate w.r.t $x_1$,

We'll have the same process algorithmically:

- Apply Trap Rule in $x_2$
- Then Trap Rule in $x_1$.

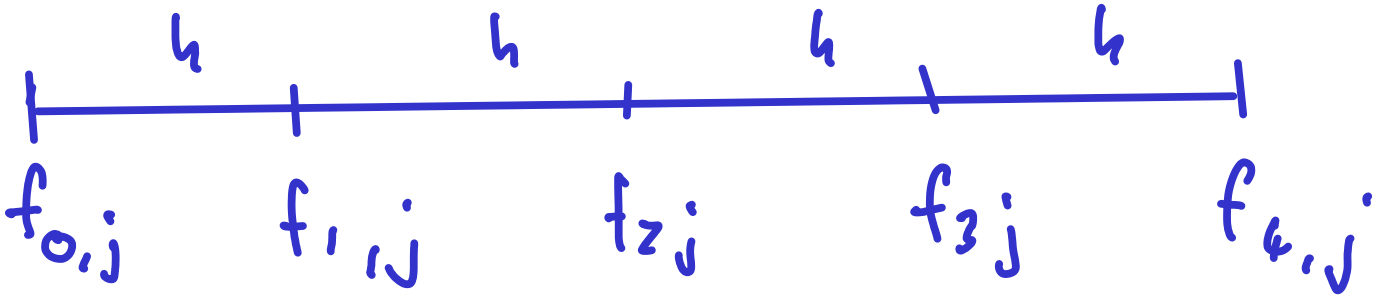
**Derivation**




$$\text{TR}: \quad \frac{h}{2}\left( f_{i,0} + 2 f_{i,1} + 2 f_{i,2} + f_{i,3} \right)$$

## Derivation

Next we apply the Trap Rule in the $x_1$ ~~direction~~ direction




$$TR : \frac{h}{2}\left(f_{0,j} + 2f_{1,j} + 2f_{2j} + 2f_{3j} + f_{4,j}\right).$$

**Derivation**

Putting this together we get the 2D Trap Rule

$$\frac{h^2}{4}\left(\sum_j \left(f_{0,j} + 2\sum_{i=1}^{N} f_{i,j} + f_{N,j}\right)\right.$$

## Implementation

$// \ (e-1)^2$

We'll implement this for estimating $\int_0^1 \int_0^1 e^{x_1+x_2} dx_1 dx_2$, with $N$ quadrature points in each direction.

00Trap2D.cpp preamble

```
double f(double x1, double x2) {  return(exp(x1+x2)); }
double ans_true = pow(exp(1.0)-1.0,2); // true value

double Trap2D(double *x1, double *x2,
              double **y, unsigned int N);
```

00Trap2D.cpp main()

```
int main(void )
{
  unsigned N = pow(2,4);        // Number of points in each direction
  double a1=0.0, b1=1.0, a2=0.0, b2=1.0;          // limits of int
  double h1, h2;                // step-size in x1 and x2
  double *x1, *x2, **y;         // quadrature points and values

  x1 = new double[N+1];
  x2 = new double[N+1];

  h1 = (b1-a1)/double(N);
  h2 = (b2-a2)/double(N);
  for(unsigned i = 0; i < N+1; i++)
  {
    x1[i] = a1+i*h1;
    x2[i] = a2+i*h2;
  }
```

1 DMA.

x1[0] = a1
x1[1] = a1 + h
x1[2] = a1 + 2h
. . . .

00Trap2D.cpp main() continued

```
y = new double * [N+1];
for(unsigned i = 0; i < N+1; i++)
  y[i] = new double[N+1];

for (unsigned i=0; i<N+1; i++)
  for (unsigned j=0; j<N+1; j++)
    y[i][j] = f(x1[i], x2[j]);

double est1 = Trap2D(x1, x2, y, N);
double error1 = fabs(ans_true - est1);

std::cout << "N=" << N << " | est=" << est1
          << " | error = " << error1 << std::endl;
```

2D DMA.

$y_{ij} = f(x_{1i}, x_{2j})$

00Trap2D.cpp main() last part

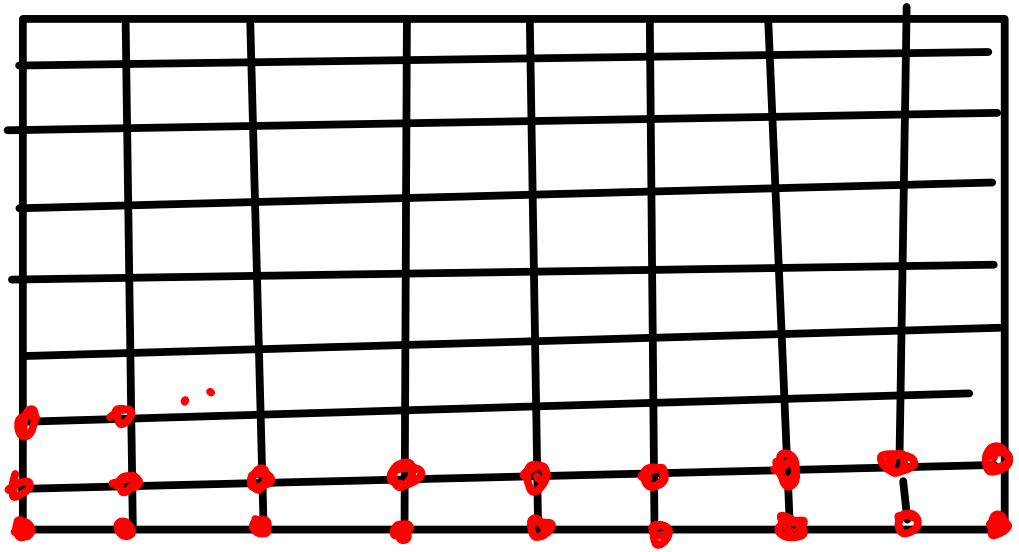
```
  // De-allocate memory
  delete [] x1;
  delete [] x2;
  for(unsigned i = 0; i < N+1; i++)
    delete [] y[i];
  delete [] y;

  return(0);
}
```

00Trap2D.cpp Trap2D()

```
double Trap2D(double *x1, double *x2, double **y,
              unsigned N)
{
  double Q, h1 = (x1[N]-x1[0])/double(N),
    h2 = (x2[N]-x2[0])/double(N);

  Q = 0.25*(y[0][0] + y[N][0]   // 4 corners
            + y[0][N] + y[N][N]);

  for (unsigned k=1; k<N; k++)   // 4 edges (not including corners)
    Q += 0.5*(y[k][0] + y[k][N]
              + y[0][k] + y[N][k]);

  for (unsigned i=1; i<N; i++) // All the points in the interior
    for (unsigned j=1; j<N; j++)
      Q += y[i][j];

  Q *= h1*h2;
  return(Q);
}
```

# 3. Preview of Labs 5 and 6

- **Lab 5** (this week)
  - Implement Simpson's Rule and Boole's Rule in 1D;
  - Verify convergence using Python/NumPy/Jupyter.
- **Lab 6** (next week)
  - Extend Simpson's Rule to 2D;
  - Compare with Monte Carlo.



Finished here

# 4. Encapsulation

**Encapsulation (aka Object Oriented Programming)**

**Idea:** create a single entity in a program that combines data with the program code (i.e., functions) which manipulate that data.
In C++, a description/definition of such entities is called a **class**, and an instance of such an entity is called an **object**.

That is, like a *variable* is a single instance for a `float` (for example), then an *object* is a single instance of a `class`.

A class should be thought of as an **Abstract Data Type** (ADT): a specialised type of variable that the user can define.

There are many important examples of "built-in" C++ classes, such as `string`, and objects, such as `cin` and `cout`. But we'll leave those until later, and first study how to make our own.

In **object oriented programming** the classes we define have two types of components:

- **data members** (also called *fields*, *attributes* or *properties*). These are usually variables of arrays.
- **function members**, which are also called **methods**. These are used to manipulate the data members.

**Note:** In CS319 when we say "**method**" we always mean a function which is a member of a class.

# 4. Encapsulation `public` V `private`

*The next bit is really important: not just to C++, but for writing robust scientific computing code.*

Within an object, methods and data members may be either

- **Private**: accessible only to another part of that object, or
- **Public**: other parts of the program can access it even though it belongs to a particular object. The public parts of an object provide an **interface** to the object for other parts of the program.

It is referred to a **"data hiding"**, an important concept in software design.

## Example – MyStack (V1.0)

In C++, *encapsulation* is implemented using the `class` keyword. The example we'll consider is a **stack** – a *LIFO* (Last In First Out) queue.

. . . . . .

*There is already a C++ implementation of a `stack`. It is part of the* **Standard Template Library (STL)**. *We reinvent the wheel here only because it is a nice example that includes most of the key concepts associated with classes in C++. We will study the STL later in CS319.*

## Example – MyStack (V1.0)

The name of our class will be `MyStack`. It will permit two primary operations:

- an item may be added to the top of the stack: `push()` ;
- an item may be removed from the top of the stack: `pop()`.

These then are our interfaces to the stack. Hence these methods will be **public**.

## Example – MyStack (V1.0)

For the stack itself, the following must be maintained:

- an array containing the items in the contents;
- a counter/index to the top of the stack.

These are *private* to the class.

We choose this example because it is obvious that

- push() and pop() are the interfaces to the object–they are declared as *public*;
- the contents of the stack, and the counter of the number of objects in it, need only be visible to the object itself; hence they are *private*.

In our example there is also a public function to initialise the stack.

for now...

New key words: class, private, public

The basic syntax for defining a class:

```
class class-name {
private:
    ...     // private methods and variables
public:
    ...     // public methods and variables
};
```

*class-name* becomes a new object type—one can now declare objects to be of type *class-name*.

This is only a declaration. Therefore,

- functions are not defined, though the prototype is given,
- variables are declared but are not initialised,
- the declaration block is delineated by { and }, and terminated with a semicolon.

As mentioned our class has two `private` members

- `contents`: a *`char`* array of length `MAX_STACK` the array containing the stacked items.
- `top`: an *`int`* that stores the number of items on the stack.

It has three `public` member functions:

(a) `init()` sets the stack counter to 0. No arguments or return value.

(b) `push()` adds an item to the stack. One argument: the character to be added.

(c) `pop()` takes no argument but returns the removed item.

01MyStack.cpp preamble

```
#define MAX_STACK 10

// Class definition of MyStack
class MyStack {
private:
  char contents[MAX_STACK];
  int top;
public:
  void init(void );
  void push(char c);
  char pop(void );
};
```

We'll study these preprocessor directives later.

need to give code for these

we push the character c onto the stack.

we pop the item from top of the stack

To define the functions associated with a particular class we use

1. the name of the class, followed by
2. the *scope resolution operator* `::` , followed by
3. the name of the function.

We now define the three (public) functions: `init()`, `push()` and `pop()`.

The `init()` is required only to set the value of `top` to zero:

`01MyStack.cpp` : `init()`

```
22 void MyStack::init(void) {
     top=0;
24 }
```

Note that we didn't have to declare the (private) variable `top`.

The push() function takes as its only argument a single character. It adds the character to the stack and increments the index to the top of the stack.

01MyStack.cpp : push()

```
26 void MyStack::push(char c) {
     contents[top]=c;
28   top++;
   }
```

← add c to top of stack.

increment the counter.

The pop() function doesn't take any arguments ( void). It removes the item from the stack by returning the top entry and decrementing top.

01MyStack.cpp : pop()

```
char MyStack::pop(void) {
  top--;
  return(contents[top]);
}
```

The first item in the stack is at position 0,
the second is a position 1,
the 3rd is at position 2, etc.
So when top=n then there are n items in the stack but the top one is actually located in contents[n-1].

Note : top-- is short for top = top-1
i.e., top -= 1

Now that our class MyStack has been declared, and its functions defined, we can declare objects to be of type MyStack, e.g.,

```
MyStack s1, s2;
```

We can refer to the functions s1.pop() and s2.push(c), say, because these are public members of the class. We cannot refer to s1.top as this variable is private to the class and is hidden from the rest of the program.

. . . . . .

To use the objects, we could have a main() function that behaves as follows:

- Declare and initialise a MyStack object s;
- Push the characters 'C','S','3','1','9' onto the stack;
- The stack's contents are popped and output to the console using cout.

01MyStack.cpp : main()

```
int main(void ) {
  MyStack s;

  s.init();

  s.push('C');
  s.push('S');
  s.push('3');
  s.push('1');
  s.push('9');

  std::cout << "Popping ... " << std::endl;

  std::cout << s.pop() << std::endl;
  std::cout << s.pop() << std::endl;
  std::cout << s.pop() << std::endl;
  std::cout << s.pop() << std::endl;
  std::cout << s.pop() << std::endl;

  return (0);
}
```

Sets s.top = 0

# 6. Constructors

Suppose we wanted to change the `MyStack` class so that the user can choose the maximum number of elements on the stack...

In the example above, the function `init()` is used explicitly to initialise the variable `top`. However, there is an initialisation mechanism called a **Constructor** that is built into the concept of a class.

**CONSTRUCTOR**

A **Constructor** is a public member function of a class

- that shares the same name as the class, and
- is executed whenever a new instance of that class is created.

ie a new object.

Constructors may contain any code you like; but it is good practice to only use them for initialization.

As an example, we'll change the declaration of the `stack` class as shown here:

```
class MyStack {
public:
  MyStack(void); // Constructor. No return type
  void push(char c);
  char pop(void);
private:
  char contents[MAX_STACK];
  int top;
};
```

replaces the init() function.

We then replace the `init()` function with:

```
MyStack::MyStack(void )
{
  top=0;
}
```

constructor name.

Class name

*Note that the constructor as no explicit return type.*

Now whenever an objects of type `MyStack` is created, e.g., with

`MyStack s;`

the function `s.MyStack()` is called automatically – and *`s.top`* is set to zero.

Complementing the idea of a **constructor**, we have a **destructor**: that is a function that is automatically called when ...

- for a local object – whenever it goes out of scope,
- for a global object – when the program ends.

The name of the destructor is the same as the class, but preceded by a tilde.

~MyStack()

However, for the versions of the `MyStack` class we've seen so far, there is nothing for a destructor to do. So, we'll have one final version of this class.

# 7. MyStack (Final version)

**MyStack: final version**

We now make some final modifications to the MyStack class. The main idea is to allow for the stack to have a maximum size that is chosen by the user, which makes it more flexible (previously it was of size MAX_STACK=10, which is very limiting.

To do that, we'll have to use **Dynamic Memory Allocation**.

Moreover, this allows us to demonstrate two things:

(a) We can have multiple constructors. In this case, one will be of default size, and the other user-chosen;

(b) A destructor, which is responsible for deallocation of memory.

For full implementation, see 02MyStackConstructor.cpp.

# 7. MyStack (Final version) New class definition

Here is the new version of the class definition. Note:

- contents is now a pointer;
- there are two MyStack() methods
- the new ~MyStack method

```
class MyStack {
private:
  char *contents;
  int top;
public:
  MyStack(void );     // default constructor
  MyStack (unsigned int StackSize);
  ~MyStack(void );    // destructor
  void push(char c);
  char pop();
};
```

top should be unsigned int too.

Code for the constructors[1]

```
MyStack::MyStack(void)
{
  top=0;
  contents = new char[MAX_STACK];
}

MyStack::MyStack(unsigned int StackSize)
{
  top=0;
  contents = new char[StackSize];
}
```

Default constructor.

[1] This is for illustration. Better again: use one constructor, but with a default argument value.

```
MyStack::~MyStack()
{
  delete [] contents;
}
```